

1 introduction

topics to be covered:

- string tokenizer
- file i/o
- command line argument
- example: document classification

it will be desirable to run this tutorial using a linux terminal – locally or remote.

2 string tokenizer

- string.h has a string tokenizer function `strtok`. its declaration looks like `char * strtok(char *input, const char *delim)`.
- it takes the string to be tokenized in the first argument in the first call. the set of delimiting characters is listed also as a string in the second argument and that is what determines the end of a token.
- in second or subsequence calls, the first argument changes to `NULL`. the delim may stay the same or change. the compiler upon seeing the null first argument, continues tokenizing from the immediate next character of the end of previous token.
- in both cases, it returns the start of token. tokenizing is done as follows.
 - any leading delimiting chars are skipped
 - if the first non-delimiting char is not `'\0'`, it is the start of next token and to be returned
 - keep on scanning any sequence of non-delimiting chars until the first delimiting char is reached. change that to `'\0'` and return the start of token.
 - if the first non-delimiting char is `'\0'` at any point, no more tokens are left and `NULL` is returned to indicate this.

Example: let us tokenize a piece of text into words. a word may terminate with the following chars: `, ; ! , , ? , . .` these will be our delim. suppose our input is:

```
char s[] = "!!!isnt_c_programming_fun?_one_can_access_memory!!!"
```

- the first `strtok` call may look like:

```
char *token = strtok(s, ",;!.");
```

- it will skip the leading exclamations. the first non-delim char is the `i`. its address will be returned. the first token ends at the space char after the `t` in `"isnt"`. that space char will be changed to `'\0'`. so, `s` will look like:

```
"!!!isnt\0c_programming_fun?_one_can_access_memory!!!"
```

- now we can do various things exploiting the null termination. we can print `token` with `printf %s`. it will print `"isnt"`. if we have a buffer `char buf[100];`, we can do `strcpy(buf, token)` to save `"isnt"` into buffer.
- the second `strtok` call will look like:

```
char *token = strtok(NULL, ",;!.");
```

- compiler internally remembers that it has seen till the space after `isnt`. so, it will start scanning from the next char 'c'. that is the start of second token to be returned. immediately there is a space which is in `delim`. so, the space will be changed to null char and the token will be returned.

```
!!!!isnt\0c\0programming_fun?_one_can_access_memory!!!!
```

as before, we can print the second token or save it into a buffer.

- in fact we can call `strtok(NULL, ",;?.");` is a loop to get all the subsequent tokens one by one in a similar manner: `programming, fun, one, ...`
- in the end, start address of "memory" will be returned as the last token. `s` will change to

```
!!!!isnt\0c\0programming\0fun\0_one\0can\0access\0memory\0!!!!
```

one more call will skip the trailing exclamations and encounter the end of string null character. since no more tokens can be found, `strtok` will return the `NULL` pointer. so we can write `while(token != NULL) {...}` to process all tokens in a loop.

it is important to note that `strtok` permanently changes strings. the input strings must be writable – string literal such as `char *s = "hello, world!"` are read-only and can't be passed into `strtok`. also, there could be situations where `strtok` is not appropriate for tokenizing such as the `rpn` calculator and we may need to write our own tokenizer from scratch using basic operations like `getchar`.

3 C programming in Linux

we will familiarize ourselves with basic c programming in linux. in particular, the following:

- basic editing with `gedit` or `vim`
- `gcc -o file file.c` or `gcc file.c` to compile in terminal
- `./file` or `./a.out` to run
- checking out man pages. `man fprintf` etc.

4 file i/o

so far we have taken input from keyboard (prutor's input box) and written into monitor. however, C programs can read from and write `ascii` files. it can only be done in linux or linux like systems such as `macos` and not in `prutor` or `windows`. one can open a file using the `fopen` function.

```
1 FILE *fp_in = fopen("input.txt", "r");
2 FILE *fp_out = fopen("output.txt", "w");
```

Files are handled using a data type called file pointers. `input.txt` must be in the same folder as the C source file. `output.txt` file will be created/over-written. "r" opens in read mode, "w" opens in write mode. if the files can't be opened in the requested mode or can't exist, these will be assigned `NULL`. once opened, one can print and scan using various functions as follows.

`stdin` and `stdout` are constant file pointers denoting the keyboard and monitor respectively. in the following, `fp_in` is a file pointer that points to a file opened in read mode. `fp_out` is a file pointer that points to a file opened in write mode.

- `fprintf(fp_out, ...)` is identical to `printf(...)` except it writes into the file pointed by `fp_out`.
- `fscanf(fp_in, ...)` is identical to `scanf(...)` except it reads from the file pointed by `fp_in`.

- `fgetc(fp_in)` or `getc(fp_in)` are identical to `getchar(...)` except it reads one char from file pointed by `fp_in`.
- `fputc(c,fp_out)` or `putc(c,fp_out)` are identical to `putchar(c)` except it writes `c` into file pointed by `fp_out`.
- `fgets(s, MAX, fp_in)` reads a line (i.e. `'\n'` terminated sequence of chars) at most `MAX - 1` char long into the string `s` from file pointer `fp_in`. `s` must be allocated/static char array large enough. then it appends a `'\0'` char in the end in `S`.
- `fputs(s, fp_out)` writes string `s` into `fp_out`.
- `ungetc(c, fp_in)` un-reads i.e. puts back the char `c` into `fp_in`. we have used `ungetc(c, stdin)` before.
- `fprintf(stdout,...)` is `printf(...)`. `fscanf(stdin,...)` is `scanf`. `fgetc(stdin)` is `getchar()`. `fputc(c, stdout)` is `putchar(c)`. `fgets(s, max, stdin)` reads a line from standard input. `fputs(s, stdout)` writes `s` into standard output.

note the file pointer argument varies from in front or in the end. detailed information about any function can be obtain using the man command. it is also available in the appendix B of textbook. after file operations are done, files can be closed using the `fclose` function.

```
1 fclose(fp_in);
2 fclose(fp_out);
```

we can go back to earlier programs that we have written using `stdio.h` functions such as `printf`, `scanf`, `getchar`, `putchar`. it is easy to change them so that they now read from / write to files instead of standard i/o. simply open the file(s) in proper mode and use one of the above functions as appropriate. an example of copying one ascii file into another is given below. an ascii file is basically a sequence of characters as far as a c program is concerned.

```
1 #include <stdio.h>
2 int main() {
3     int c;
4     FILE *fp_in = fopen("input.txt", "r");
5     FILE *fp_out = fopen("output.txt", "w");
6     if(fp_in == NULL || fp_out == NULL) {
7         printf("file_can't_be_opened\n");
8         return 1;
9     }
10    while((c = fgetc(fp_in)) != EOF)
11        fputc(c, fp_out);
12    fclose(fp_in);
13    fclose(fp_out);
14 }
```

5 command line argument

C programming in linux allows us to pass arguments while running the program. this is very helpful. for example, in the previous code, we were copying file `input.txt` into the file `output.txt`. now, suppose we want to copy another file to another output file. we need to go to the source code and change the name of these two files and compile, run our program. instead it is much more convenient to run it as follows.

```
1 ./a.out input.txt output.txt
```

command line arguments allows us to do exactly that. what ever is written after `./a.out` before pressing the enter could be fed to our main functions using two arguments:

- an int parameter denoting the total number of arguments. typically called `argc`.

- an array of char pointers denoting each of the argument. typically called argv. argv[0] is the name of the program ./a.out or something else. argv[argc] is always NULL

So, in the above, argc is 3. argv[0], argv[1], and argv[2] are pointers to "./a.out", "input.txt", "output.txt" respectively. argv[3] is NULL.

Given this simple convention, it is now easy to modify our code to make it more convenient as discussed above.

```
1 #include <stdio.h>
2 int main(int argc, char *argv[]){
3     if(argc != 3){
4         printf("wrong_use\n");
5         return 1;
6     }
7     int c;
8     FILE *fp_in = fopen(argv[1], "r");
9     FILE *fp_out = fopen(argv[2], "w");
10    if(fp_in == NULL || fp_out == NULL){
11        printf("file_can't_be_opened\n");
12        return 1;
13    }
14    while((c = fgetc(fp_in)) != EOF)
15        fputc(c, fp_out);
16    fclose(fp_in);
17    fclose(fp_out);
18 }
```

6 example: document classification

!!! IMP

in this exercise, we will build an elementary document classifier using the bag-of-words technique. A document classifier classifies an input document into one among a set of pre-determined classes, say: Sports, Politics, Tech. Each such class will have its own set of keywords. For simplicity, assume no two classes share a keyword. the classes are completely specified using an input file say "classes.txt" that looks as follows.

```
Sports: match, team, player, score, game
Politics: election, vote, government, policy, minister
Tech: AI, code, software, computer, algorithm
```

That is, one class per line: class name, colon, space, then the list of keywords separated by a comma and a space. it is assumed that each line is at most 99 char long. it is assumed that there are at most 5 classes and each class has at most 9 keywords.

The document "doc.txt" itself looks as follows. it is assumed line and words are at most 499 char and 19 char long respectively.

```
1 The team played an exciting match. Every player contributed to the game and helped the
   team score well.
2
3 Meanwhile, the government announced a new policy after the election. The minister
   encouraged citizens to vote responsibly.
4
5 In another development, the AI software uses a new algorithm. The computer code was
   improved to make the system faster. AI is becoming important in many fields.
```

The classification works using a simple rule as follows. first split the document into a sequence of words using the delimiters " , ; ? . ". Then obtain the count for each keywords for each classes. Ignore cases while matching keywords. The class having the largest count is the predicted class of the document. The prediction is summarized into an output file "output.txt" as follows.

```

1 Document: doc.txt
2
3 Top Matching Words:
4 Sports -> match(1), team(2), player(1), score(1), game(1)
5 Politics -> election(1), vote(1), government(1), policy(1), minister(1)
6 Tech -> AI(2), code(1), software(1), computer(1), algorithm(1)
7
8 Class Predictions: confidence
9 Sports: 6 / 17 = 35.3%
10 Politics: 5 / 17 = 29.4%
11 Tech: 6 / 17 = 35.3%

```

in case of no matching keywords at all, print some error message for predicted class. the program is supposed to be run as:

```

1 ./a.out classes.txt doc.txt output.txt

```

Here is the complete code.

6.1 main function

Let us first discuss the main function and the declarations.

- we will use various file I/O functions from `stdio.h`. `strtok` and `strdup` from `string.h`. `free` from `stdlib.h` and `tolower` from `ctype.h`
- we assumed 5 classes and 9 keywords at maximum for each class. the parameters are one more to accommodate termination with NULL pointers. similarly, we define the maximum characters per line in class file and doc file.
- main function first checks for proper usage using `argc` and reports error otherwise.
- it then declares an array of pointers called `classes`. each index `i` of this array will be allocated and copied with the `i`-th class name. after filling all class names, the next index of this array will be assigned NULL.
- next, there is a 2D array of pointers called `keywords`. each `[i][j]` index of this 2D array will hold the `j`-th keyword of the `i`-th class after proper allocation and copy. after filling keywords for all the `n` classes say, `keywords[i][0]` will be assigned NULL.
- the above NULL termination property helps us to denote end of classes or keywords implicitly. we would need to carry around a lot of counts otherwise. it is similar in spirit why strings are `'\0'` terminated in C, or why `argv[argc]` is NULL pointer or why we use `-1` to denote end of a sequence of positive integers.
- `process_classfile` function reads the class information file with keywords and copies the class names into the `classes` array and keywords into the `keywords` 2D array.
- then we initialize counts for each keyword and classes and assign them `{}` to set each count to 0.
- `process_docfile` function reads the document and tokenizes it into a sequence of words. each word is matched with the set of keywords and the counts are updated upon matching.
- next we generate class counts using the keyword counts and use these two counts to generate our report.
- finally, we free the strings allocated by `strdup` at the addresses given by the pointers in the `classes` and `keywords` arrays.

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <ctype.h>
5
6 //one more than max to accommodate NULL
7 #define MAX_CLASSES 5
8 //one more than max to accomodate NULL
9 #define MAX_KEYWORDS 10
10
11 //one more than max to accommodate '\0'
12 #define MAX_CLASS_LINE 100
13 //one more than max to accommodate '\0'
14 #define MAX_DOC_LINE 500
15
16 int main(int argc, char *argv[]){
17     if(argc != 4){
18         printf("Usage:_%s_<classes.txt>_<document.txt>_<output.txt>\n", argv[0]);
19         return 1;
20     }
21
22     char *classes[MAX_CLASSES];
23     //keywords[i][j] gives the j-th keyword of the i-th class
24     char *keywords[MAX_CLASSES][MAX_KEYWORDS];
25
26     //first build the classes.
27     process_classfile(argv[1], classes, MAX_CLASSES, keywords, MAX_KEYWORDS);
28
29     //declare counters for each keyword. start all with 0
30     //last class and last keywords are NULL
31     //declare class counters and set to 0s
32     int keyword_count[MAX_CLASSES - 1][MAX_KEYWORDS - 1] = {};
33     int class_count[MAX_CLASSES - 1] = {};
34
35     //now, match keywords and update counters
36     process_docfile(argv[2], classes, keywords, keyword_count);
37
38     //compute class counts from keyword counts
39     compute_class_counts(class_count, keyword_count, keywords);
40
41     //finally generate report based on keyword count
42     generate_report(argv[2], argv[3], classes, keywords, class_count, keyword_count);
43
44     //free strdup allocations
45     for(int i = 0; keywords[i][0] != NULL; i++){
46         free(classes[i]);
47         for(int j = 0; keywords[i][j] != NULL; j++)
48             free(keywords[i][j]);
49     }
50
51     return 0;
52 }

```

6.2 process_classfile function

this functions opens the class file in "r" mode. reads line by line in a loop. each line is supposed to have the class name and its keywords. these are extracted using appropriate use of strtok and saved into the classes array and the keywords array.

we use the function `strdup` (string duplicate) function from `string.h` to duplicate a string.

```
1 char *strdup(const char *s);
```

this function takes a `'\0'`-terminated string `s` as its argument and makes a copy/duplicates it and then returns a pointer to the copy. it is equivalent to first allocating `strlen(s) + 1` bytes and then doing `strcpy` on the allocated address and then returning the address. that is why it is important for us to free the memory for all these allocations in the end in the main function.

also, note that the array of class names is ended with a `NULL` pointer to implicitly denote the end. similarly, the `keywords[num_classes][0]` is also assigned `NULL` pointer to denote no more row of keywords. finally, within each row of keywords corresponding to some class, we have terminated with `NULL` after pointing to the last keyword. this of course assumes that there is no class devoid of any keyword, which is true.

```
1 void process_classfile(char *classfile, char *classes[], int max_classes, char *
    keywords[][MAX_KEYWORDS], int max_keywords) {
2     //tokenize classfile to build keywords and classes.
3     FILE * fp = fopen(classfile, "r");
4     if(fp == NULL) {
5         printf("error_opening_class_file\n");
6         exit(1);
7     }
8     //read line by line and store
9     int num_classes = 0, num_keywords;
10    char line[MAX_CLASS_LINE], *token;
11    while(num_classes < max_classes && fgets(line, MAX_CLASS_LINE, fp) != NULL) {
12        //retrieve the classname
13        token = strtok(line, ":");
14        classes[num_classes] = strdup(token);
15
16        //now retrieve the keywords for this class
17        num_keywords = 0;
18        while(num_keywords < max_keywords && (token = strtok(NULL, ",_\n")) != NULL) {
19            //fgets keeps the newline, also a delimiter
20            keywords[num_classes][num_keywords] = strdup(token);
21            num_keywords++;
22        }
23        //NULL termination to denote end
24        keywords[num_classes][num_keywords] = NULL;
25        num_classes++;
26    }
27
28    //terminate classes and keywords with NULL pointer to denote end
29    classes[num_classes] = NULL;
30    keywords[num_classes][0] = NULL;
31    fclose(fp);
32 }
```

6.3 process_docfile and search_keys functions

we just open the docfile. using `strtok`, we obtain words one by one. then match with the set of keywords using the `search_keys` functions. if a match is found, the corresponding counter in `keyword_count` is incremented.

the `search_keys` function is also given below. it simply makes a pass over each keyword for each class. this is where `NULL` termination help us for looping over. for matching, the built-in `strcmp` will not work as we need to ignore case – so we write our own function.

```

1 void process_docfile(char *docfile, char *classes[], char * keywords[][MAX_KEYWORDS],
  int keyword_count[][MAX_KEYWORDS - 1]){
2  //tokenize words. match words. count matches.
3  FILE *fp_in = fopen(docfile, "r");
4  if(fp_in == NULL){
5      printf("error_opening_document_file\n");
6      exit(1);
7  }
8
9  //tokenize to get words one by one
10 int keyword_class, keyword_index;
11 char line[MAX_DOC_LINE], *token;
12
13 while(fgets(line, MAX_DOC_LINE, fp_in) != NULL){
14     token = strtok(line, ",;_?.\n"); //fgets includes \n
15     while(token != NULL){
16         search_keys(keywords, token, &keyword_class, &keyword_index);
17         if(keyword_class >= 0) //found
18             keyword_count[keyword_class][keyword_index]++;
19         token = strtok(NULL, ",;_?.\n");
20     }
21 }
22 fclose(fp_in);
23 }
24
25 void search_keys(char * keywords[][MAX_KEYWORDS], char *word, int *matched_class, int
  *matched_key){
26     //search for word into the set of keywords and implicitly return
27     int i, j;
28     //recall last + 1 class has the first keyword NULL
29     for(i = 0; keywords[i][0] != NULL; i++){
30         j = 0;
31         while(keywords[i][j] != NULL){
32             //use NULL termination to traverse keywords
33             if(case_insensitive_strcmp(keywords[i][j], word) == 0){
34                 *matched_class = i;
35                 *matched_key = j;
36                 return;
37             }
38             j++;
39         }
40     }
41     //not a keyword
42     *matched_class = -1;
43     *matched_key = -1;
44 }
45
46 int case_insensitive_strcmp(char *s, char *t){
47     while(*s && *t && tolower(*s) == tolower(*t)){
48         s++;
49         t++;
50     }
51     return *s - *t;
52 }

```

6.4 compute_class_counts and generate_report functions

it is easy to get the class counts from the keyword counts – just sum up all keyword counts for a particular class. again, we use the NULL termination to iterate over all classes and all keywords. same is true for generating the

report. notice the use of file I/O functions in the later function.

```
1 void compute_class_counts(int class_count[], int keyword_count[][MAX_KEYWORDS - 1],  
    char * keywords[][MAX_KEYWORDS]) {  
2     //loop over classes  
3     int sum;  
4     for(int i = 0; keywords[i][0] != NULL; i++){  
5         //loop over keywords and sum.  
6         sum = 0;  
7         for(int j = 0; keywords[i][j] != NULL; j++)  
8             sum += keyword_count[i][j];  
9         class_count[i] = sum;  
10    }  
11 }
```

```

1 void generate_report(char *docfile, char *outfile, char *classes[], char * keywords[][
    MAX_KEYWORDS], int class_count[], int keyword_count[][MAX_KEYWORDS - 1]){
2     //now generate report
3     FILE *fp_out = fopen(outfile, "w");
4     if(fp_out == NULL){
5         printf("error_opening_output_file\n");
6         exit(1);
7     }
8
9     //write report to fp_out as in desired format
10    fprintf(fp_out, "Document:_%s\n\nTop_Matching_Words:\n", docfile);
11    int j, first;
12    int total_keyword_count = 0;
13    for(int i = 0; keywords[i][0] != NULL; i++){
14        fprintf(fp_out, "%s->", classes[i]);
15
16        //now make a pass over all keywords and print counts
17        j = 0;
18        first = 1;
19        while(keywords[i][j] != NULL){
20            if(keyword_count[i][j] == 0){
21                //omit
22                j++;
23                continue;
24            }
25            else if(first){
26                fprintf(fp_out, "%s(%d)", keywords[i][j], keyword_count[i][j]);
27                first = 0;
28            }
29            else{
30                fprintf(fp_out, ",_%s(%d)", keywords[i][j], keyword_count[i][j]);
31            }
32            total_keyword_count += keyword_count[i][j];
33            j++;
34        }
35        fprintf(fp_out, "\n");
36    }
37
38    fputs("\nClass_Predictions:_confidence", fp_out);
39    if(total_keyword_count > 0){
40        for(int i = 0; keywords[i][0] != NULL; i++)
41            fprintf(fp_out, "\n%s:_%d/_%d=_.1f%%", classes[i], class_count[i],
                total_keyword_count, 100.0 * class_count[i] / total_keyword_count);
42    }
43    else
44        fputs("\nno_keyword_match_--_could_not_predict\n", fp_out);
45    fclose(fp_out);
46 }

```

6.5 exercises

- change all file i/o to stdin or stdout properly so that doc and report can be fed and run in the input box of prutor.
- output the top matching words sorted high to low. output the class predictions sorted high to low. currently, both follow the order as in the classes.txt file.